

inHunt 灵觅 — 中期汇报文档

移动应用开发大作业 | 2026年5月14日

仓库地址: github.com/Chris-behind-door/Inspirational_Writer

一、项目主要界面设计

1.1 整体架构

inHunt 采用底部 Tab 导航结构 (Expo Router file-based routing)，共四个主页面：

Tab	名称	功能定位
灵感捕捉	Inspire	AI 生成灵感卡片，支持角色/情节/世界观/对话四种类型
灵感记录	Collection	灵感收藏与管理，支持展开查看和删除
故事管理	Story	从灵感到故事的创作工作空间（规划中）
设置	Settings	API 配置、模型参数、写作偏好

1.2 界面截图

灵感捕捉页

- 四种灵感类型卡片（角色灵感 / 情节转折 / 世界观构建 / 对话片段）
- 选中高亮 + 补充描述输入框
- AI 生成按钮 + 加载态 + Markdown 渲染结果
- 原文切换：支持长按选中复制原始文本
- 生成结果自动保存至灵感记录
- 未配置 API 时显示引导文案

灵感记录页

- 灵感列表（类型标签 + 提示摘要）
- 展开/折叠查看 Markdown 渲染的完整内容
- 原文切换：查看/复制原始 Markdown 文本
- 删除确认（Alert）
- 空状态引导

设置系统

- **API 配置**：预设管理（增删改选）、连接测试、发送测试消息
- **模型参数**：Temperature / Top P / Frequency Penalty / Presence Penalty 滑块调节
- **写作偏好**：文风偏好 + 补充指令文本框

运行截图见附件视频。

二、项目关键页面实现方案

2.1 已完成页面

灵感捕捉 (Inspire)

- **实现方式：**
- 从 `configStore` 读取当前 API 预设 (Base URL / Key / Model) 、模型参数和写作偏好
- 四种预设灵感类型各有专用 System Prompt
- 用户偏好 (文风、补充指令) 自动注入 System Prompt
- 调用 OpenAI-compatible `/chat/completions` 接口
- 生成结果以自研 Markdown 渲染器展示，自动存入 AsyncStorage 持久化
- 支持 原文切换，长按选中复制原始文本
- **已实现功能：** 类型选择、自定义补充、AI 调用、Markdown 渲染、原文切换、自动保存

灵感记录 (Collection)

- **实现方式：**
- 使用 `useFocusEffect` 在页面获得焦点时从 AsyncStorage 加载
- 列表展示，支持展开/折叠查看 Markdown 渲染详情
- 支持 原文切换查看原始文本
- Alert 确认删除
- **已实现功能：** 列表展示、展开详情、Markdown 渲染、原文切换、删除记录、空状态

Markdown 渲染 (自研组件)

- **实现方式：**
- 基于 `simple-markdown` 解析器，自研 `MarkdownRenderer` 组件
- 支持标题 (H1-H4) 、粗体、斜体、删除线、行内代码、代码块

- 有序/无序列表、嵌套列表
- 引用块（蓝色左边框 + 浅蓝背景）
- 针对 `simple-markdown` 的已知缺陷做了预处理修复（标题后空行、emoji 标题等）
- **技术选型：** 纯 React Native View/Text 渲染，无第三方 Markdown 渲染库依赖

设置系统

- **实现方式：**
- 全局 `configStore.ts` 管理所有持久化数据（AsyncStorage）
- API 预设支持增删改选 + 实时连接测试 + 测试消息发送
- 模型参数使用 Slider 组件调节，实时保存
- 写作偏好文本输入，保存到 AsyncStorage
- **已实现功能：** 完整的 API 配置闭环、参数调节、偏好设置

2.2 未完成页面及计划

故事管理 (Story)

- **计划实现方案：**
- 从灵感记录中选择素材，组合成故事大纲
- 分章节编辑器（TextInput + 自动保存）
- AI 辅助续写功能（基于已有章节内容调用 API）
- **技术选型：** AsyncStorage 持久化故事数据，复用现有 AI 调用模式

灵感连接

- **计划方案：** 允许用户手动关联多条灵感（如"角色A"关联"情节B"），形成灵感图谱
 - **展示方式：** 使用 React Native 原生 View 绘制简单关系图（或引入轻量图形库）
-

三、项目主要模块设计

3.1 前端模块架构

```
app/  
├─ (tabs)/  
│   ├─ inspire.tsx           # 灵感捕捉: AI 调用 + Markdown 渲染  
│   ├─ collection.tsx        # 灵感记录: 列表展示 + 本地存储  
│   ├─ story.tsx             # 故事管理 (规划中)  
│   └─ settings.tsx          # 设置入口导航  
├─ api-config.tsx            # API 预设管理 (增删改选 + 连接测试)  
├─ model-settings.tsx        # 模型参数调节  
├─ preferences.tsx           # 写作偏好设置  
├─ api-docs.tsx              # API 使用文档  
└─ _layout.tsx               # 根布局 (Stack 导航)  
components/  
└─ MarkdownRenderer.tsx      # 自研 Markdown 渲染器 (simple-markdown 驱动)  
configStore.ts                # 全局数据管理 (AsyncStorage 持久化)  
types/  
└─ simple-markdown.d.ts      # 类型声明
```

3.2 数据层设计

```
configStore.ts                # 全局数据管理  
├─ Preset[]                   # API 预设列表 (BaseUrl/Key/Model)  
├─ activePresetId             # 当前应用的预设  
├─ ModelSettings              # 模型参数 (temperature/topP/freqPenalty/presPenalty)  
└─ Preferences                # 写作偏好 (文风/补充指令)  
  
AsyncStorage Keys:  
├─ @configStore/presets  
└─ @configStore/activePresetId
```

```
└─ @configStore/modelSettings
└─ @configStore/preferences
└─ @inhunt/inspirations # 灵感记录 (InspirationItem[])
```

3.3 模块间交互逻辑

- 用户选择灵感类型
- 读取 activePreset + modelSettings + preferences
 - 拼接 System Prompt (偏好 + 类型专用提示)
 - POST /chat/completions → MarkdownRenderer 渲染
 - 自动写入 @inhunt/inspirations
 - 灵感记录页 useFocusEffect 自动刷新展示

当前无后端模块。 所有 AI 调用从前端直接发起 (OpenAI-compatible API)，数据存储使用 AsyncStorage 本地持久化。

四、项目问题梳理与解决思路

4.1 已遇到的问题

问题	状态	解决方案
Expo Router 路径类型推断报错	已解决	使用 <code>as any</code> 类型断言绕过 (Expo Router 已知问题)
API 连接测试需真实网络请求	已解决	调用 <code>/models</code> 端点测连通性 + <code>/chat/completions</code> 测功能
预设切换后表单值未联动	已解决	<code>useEffect</code> 加载时自动选中已应用的预设并填充表单
		<code>originalValues</code> 对比逻辑判断

问题	状态	解决方案
已应用预设未修改时保存按钮应禁用	已解决	
Markdown 列表渲染（ <code>* item</code> 消失）	已解决	第三方库有 bug，自研 <code>MarkdownRenderer</code> 替换
行内节点换行（粗体后文本断开）	已解决	连续行内节点合并到单个 <code><Text></code> ，块级节点分离渲染
嵌套列表不可见	已解决	<code>simple-markdown</code> 将嵌套列表包在 <code>paragraph</code> 内，递归处理
标题后紧跟列表/引用被吞掉	已解决	预处理补空行（ <code>simple-markdown</code> 已知缺陷）
emoji 破坏标题解析（ <code>### 标题</code> ）	已解决	预处理统一格式化 <code>{n}</code> 标题 空格

4.2 潜在问题与应对措施

潜在问题	应对措施
AI API 调用失败 （网络/配额/Key 过期）	前端显示详细错误信息，引导用户检查 API 配置；后续考虑加 fallback 模型
AsyncStorage 容量限制 （灵感记录增多）	当前使用 JSON 数组存储，后续可改为分页加载 + 索引优化；或引入 SQLite
Markdown 渲染兼容性 （部分模型输出格式不标准）	提供" 原文"切换查看原始输出；预处理修复已知解析器缺陷
React Native Markdown 生态不成熟	自研组件替代第三方库，彻底控制渲染行为

潜在问题	应对措施
灵感生成质量不稳定	通过写作偏好让用户自定义风格； System Prompt 持续迭代优化

4.3 团队协作说明

自 2026 年 4 月 18 日项目初始化以来，除项目负责人外，其余组员未提交任何代码，群消息也未获回复。鉴于中期检查为项目进度评估节点，本报告基于当前实际完成内容撰写，后续将视情况与任课老师沟通组员分工调整。

附录：技术栈

类别	技术
框架	React Native + Expo (Expo Router)
语言	TypeScript
导航	Expo Router (File-based Routing)
持久化	AsyncStorage
AI 接口	OpenAI-compatible /chat/completions
Markdown 解析	simple-markdown + 自研 Renderer
UI 风格	iOS 设置页风格（#F2F2F7 背景 + 白色圆角卡片）